Zurück zum Fall

ARBEITSBESTAND UND PUBLIKATION

Eine Ablage für Forschung – eine Publikation für andere.

Elifs Frage

Lea gibt Elif Zugang zu ihrem Forschungsrepo. Elif möchte Reimformen untersuchen und fragt: „Welchen TEI-Ordner soll ich nehmen?“

Für Leas Untersuchung sinnvoll

Das Repo bewahrt die übernommenen Quellen, die vereinheitlichten Daten und die festgehaltenen Eingaben einzelner Studien. Dasselbe Buch liegt dadurch in mehreren begründeten Fassungen vor. Diese Struktur dokumentiert Leas Forschungsweg. Elif braucht dagegen einen eindeutig bezeichneten Korpusbestand für eine neue Untersuchung.

MERKEN

Ein Publikationsrepositorium ist kein veröffentlichtes Forschungsrepo!

Eine gute Forschungsablage erklärt, wie eine Studie entstanden ist. Die Datenpublikation erklärt, welchen Bestand andere für neue Studien verwenden sollen. Dafür richten wir ein eigenes kuratiertes Korpusrepo ein.

▼ Beispiel und Tipp

Beispiel: Forschungsrepo: quellen/tei/rk_1786_b0001.xml → daten/vereinheitlicht/rk_1786_b0001.xml → analysen/reimstudie/eingabe/rk_1786_b0001.xml. Publikationsrepo: genau ein maßgeblicher Korpusstand unter data/tei/rk_1786_b0001.xml.

Tipp: Publikationskonzept vor der ersten Freigabe in Git festhalten.



▼ Grafik als Text lesen

Leas Forschungsablage

1. Leas Forschungsrepo/
2. quellen/tei/
3. rk_1786_b0001.xml · übernommene Quelle
4. daten/vereinheitlicht/
5. rk_1786_b0001.xml · aufbereitete Daten
6. analysen/reimstudie/eingabe/
7. rk_1786_b0001.xml · festgehaltener Studienstand
8. Ein Forschungsweg: Quelle → Aufbereitung → Studienstand.

ZWEI ZWECKE · ZWEI ANGEBOTE

Für die Studie sinnvoll – für die Publikation anders aufbereitet

Im Forschungsrepo	Warum dort sinnvoll?	Im Publikationsrepo
quellen/tei/rk_1786_b0001.xml	Übernommene Fassung für den Vergleich mit der Aufbereitung.	Quellennachweis und Herkunft in Metadaten; kein konkurrierender Korpusordner.
daten/vereinheitlicht/rk_1786_b0001.xml	Aufbereiteter Stand für Leas Auswertungen.	Nach Prüfung der maßgebliche Bestand unter **data/tei/rk_1786_b0001.xml**.
analysen/reimstudie/eingabe/rk_1786_b0001.xml	Festgehaltene Eingabe, mit der ein Studienergebnis entstand.	Die Studie nennt Release und Auswahl. Ihr eingefrorener Ausschnitt wird nicht als zweite Korpusfassung angeboten.

Die Forschungsablage ist für ihren Zweck angemessen. Für die Publikation werden eindeutiger Einstieg, geltende Regeln und ein freigegebener Bestand ergänzt.

Die vollständige empfohlene Publikationsstruktur ansehen ↑

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Publikationskonzept vor der ersten Freigabe in Git festhalten.
2. Geprüfte Buchdateien aus daten/vereinheitlicht nach data/tei des Publikationsrepos übernehmen.
3. README, Regeln und Einstieg ergänzen; den ersten Release extern ausprobieren lassen.

Konkretes Beispiel

Forschungsrepo: quellen/tei/rk_1786_b0001.xml → daten/vereinheitlicht/rk_1786_b0001.xml → analysen/reimstudie/eingabe/rk_1786_b0001.xml. Publikationsrepo: genau ein maßgeblicher Korpusstand unter data/tei/rk_1786_b0001.xml.

Erledigt, wenn: Eine neue Nutzerin erkennt ohne Rückfrage, welche Dateien und welche Version sie untersuchen soll.

Git hilft dabei: Git hält Publikationskonzept, Daten und Korrekturen nachvollziehbar zusammen. GitHub macht README, Releases, Issues und geprüfte Beiträge zugänglich. Welche Arbeitsgeschichte öffentlich wird, entscheidet das Team bewusst.

QUALITÄT PRÜFEN

Das Skript läuft. Wo sind die Lösungen?

Eine unbemerkte Auslassung

Leas neue Aufbereitung übernimmt die Rätseltexte, verliert aber den Lösungsteil: Er war in der bisherigen TEI-Struktur in einem getrennten XML-Element erfasst und ist in den neu erzeugten TEI-Daten nicht mehr enthalten. Ihre bisherige Python-Auswertung zählt nur Rätsel und Verse: Die Zahlen bleiben gleich. Erst beim Lesen einer Seminaraufgabe fehlt die Antwort.

Den Fehler eingrenzen

Lea und Jonas vergleichen in Git die Fassungen vor und nach der Konvertierung. Dabei erkennen sie, dass die geänderte Konvertierung das XML-Element mit den Lösungen nicht übernimmt. Sie korrigieren das Verfahren und erstellen einen kleinen neuen Prüffall, der genau diesen Fehler nachstellt: Ein Rätsel mit einem getrennt ausgezeichneten Lösungsteil muss die Konvertierung vollständig durchlaufen. Der Test prüft anschließend, ob Lösung und Zuordnung erhalten geblieben sind.

MERKEN

Eine erfolgreiche Analyse beweist keine vollständigen Daten.

Wir prüfen Struktur, Beziehungen und Inhaltserhalt getrennt. Automatische Tests werden durch einen gezielten Quellenvergleich ergänzt.

▼ Beispiel und Tipp

Beispiel: Zwei Rätsel und zwei Lösungen vor der Verarbeitung; zwei Rätsel und keine Lösung danach. XML und Demo-Schema bestehen, der Bestandsvergleich meldet den Verlust.

Tipp: Vor und nach einer Transformation alle zu übernehmenden Elemente und Inhalte, IDs und Verweise vergleichen.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Vor und nach einer Transformation alle zu übernehmenden Elemente und Inhalte, IDs und Verweise vergleichen.
2. Im Pull Request den Diff und betroffene Quellstellen prüfen; unerwartete Verluste klären.
3. Den Fehlerfall als Regressionstest behalten und erst nach fachlicher Freigabe veröffentlichen.

Konkretes Beispiel

Zwei Rätsel und zwei Lösungen vor der Verarbeitung; zwei Rätsel und keine Lösung danach. XML und Demo-Schema bestehen, der Bestandsvergleich meldet den Verlust.

Erledigt, wenn: Der Prüflauf erkennt den bekannten Verlust; die Quellenkontrolle und Freigabe sind dokumentiert.

Git hilft dabei: Versionsvergleich und bezeichnete Werkzeugstände zeigen, wann Inhalte verloren gingen und welche Importe betroffen sind. Ein Regressionstest begleitet die Korrektur. Git bewahrt Änderungen; ihren fachlichen Sinn muss das Team prüfen.

VERSIONEN UND FORSCHUNGSNACHWEIS

Gestern war das Ergebnis noch anders.

Eine neue Grafik

Für ihre Publikation hat Elif den Versanteil im Korpus berechnet und eine Balkengrafik erzeugt, die ihr sehr aussagekräftig erschien. Etwas später möchte sie mit demselben Skript

dieselbe Grafik in ausdrucksstärkeren Farben erzeugen. Doch die Balken zeigen ein unerwartetes Bild: Lea hat inzwischen weitere Sammlungen aufgenommen. Elifs Skript lädt stets den aktuellen Repositorystand.

Den verwendeten Stand wiederholen

Mit dem archivierten früheren Release und demselben Skript kann Elif die ursprüngliche Grafik wiederherstellen. Sie muss die verwendete Datenversion angeben, damit andere ihre Zahlen und Schlussfolgerungen nachvollziehen können. An ihren Schlussfolgerungen ändert der erweiterte Bestand nichts; einige Zahlen fallen allerdings weniger eindeutig aus als zuvor. Elif hält nun Datenfassung, Belegauswahl und Verfahren gemeinsam fest. Lea kann weiter Bücher ergänzen, während frühere Untersuchungen auf ihren bezeichneten Datenstand zurückgreifen können.

MERKEN

Eine Untersuchung braucht einen bestimmten Datenstand.

Wir veröffentlichen geprüfte Releases und erhalten sie unabhängig vom Git-Host. Forschende nennen zusätzlich Auswahl, Extraktion und Werkzeugfassung.

▼ Beispiel und Tipp

Beispiel: Demo-v1.0: 2 von 4 Rätseln sind Verse. Demo-v1.1: 8 von 10. Werden dieselben vier alten Belege ausgewählt, bleibt ihr Anteil bei 50 Prozent.

Tipp: Nach der Freigabe einen geschützten Tag und Release mit Bestands- und Änderungsnotizen anlegen.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Nach der Freigabe einen geschützten Tag und Release mit Bestands- und Änderungsnotizen anlegen.
2. Datenpaket samt Metadaten archivieren und den konkreten Versionsnachweis in den Zitierhinweis aufnehmen.
3. Für jede Analyse Release, Auswahl, Exportregel und Skriptfassung festhalten.

Konkretes Beispiel

Demo-v1.0: 2 von 4 Rätseln sind Verse. Demo-v1.1: 8 von 10. Werden dieselben vier alten Belege ausgewählt, bleibt ihr Anteil bei 50 Prozent.

Erledigt, wenn: Eine andere Person kann aus den bezeichneten Daten und Verfahren die Untersuchung wiederholen.

Git hilft dabei: Fixierte Daten- und Werkzeugstände machen den Vergleich wiederholbar. Git dokumentiert Bestandsänderungen; deren wissenschaftliche Bedeutung muss das Team erklären.

NACHNUTZUNG UND ANNOTATION

Ein Reim hilft dem Modell auf die Sprünge.

Eine Frage aus dem Seminar

Studierende lassen ein lokales Sprachmodell gereimte Zweizeiler lösen. In einer Gruppe dieser Rätsel ist die Lösung zugleich das fehlende Reimwort im zweiten Vers. „Versteht es das Rätsel oder ergänzt es nur den Reim?“ Um das zu untersuchen, vergleicht die Gruppe die Verse mit einer getrennt gespeicherten Prosabearbeitung, in der der Reimhinweis fehlt.

Aus Nutzung wird Anschlussforschung

Elif ergänzt in den für diese Untersuchung verwendeten Daten Form- und Wortannotation. Damit kennzeichnet sie insbesondere die Rätsel, deren Lösung das fehlende Reimwort ist. Bei einem späteren Workshop werden die so aufbereiteten Daten für einen Vergleich verschiedener Sprachmodelle genutzt. Es entsteht ein separates kleines Benchmark-Korpus: Welches Modell kann sprachliche Rätsel im Deutsch vergangener Sprachstufen lösen? Ein weiteres Experiment trainiert ein Sprachmodell mithilfe der Referenzdaten nach; auf den getrennt gehaltenen Testdaten erzielt es deutlich bessere Lösungsergebnisse. Das Team entscheidet, das gesamte vorhandene Korpus für solche Aufgaben aufzubereiten. Die Daten erhalten eine zusätzliche Annotationsschicht. TEI erlaubt, diese Schicht gesondert auszuzeichnen und mit den Basisdaten zu verknüpfen, ohne deren Texte zu verändern. Die erweiterte Fassung erhält eine neue Versionsnummer; Basisrelease, Annotation und Modellversuche bleiben nachvollziehbar miteinander verbunden.

MERKEN

Neue Forschung soll auf dem Korpus aufbauen können.

Unser Ausgangspunkt ist eine getrennt versionierte Zusatzannotation. Originale, Bearbeitungen, automatische Schichten und Modellversuche bleiben eindeutig unterscheidbar.

▼ Beispiel und Tipp

Beispiel: Basis: Korpus v1.0. Zusatz: Wortannotation 1.0 mit Bezug auf dessen Tokenisierung. Modelltest: Rätseltexte in testeingaben.jsonl, Referenzantworten getrennt in loesungen.jsonl.

Tipp: Annotation mit Basisrelease, Beleg- oder Wortankern und Erzeugungsverfahren beschreiben.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Annotation mit Basisrelease, Beleg- oder Wortankern und Erzeugungsverfahren beschreiben.
2. Automatische Herkunft und Prüfstatus angeben; Basisdaten ohne Zusatzschicht anbieten.
3. Bei Modellversuchen Testeingaben und Lösungen trennen; Trainings-/Testauswahl und verwandte Belege prüfen.

Konkretes Beispiel

Basis: Korpus v1.0. Zusatz: Wortannotation 1.0 mit Bezug auf dessen Tokenisierung. Modelltest: Rätseltexte in testeingaben.jsonl, Referenzantworten getrennt in loesungen.jsonl.

Erledigt, wenn: Die Annotation ist ihrer Basis zugeordnet und kann weggelassen werden; der Versuch ist dokumentiert.

Git hilft dabei: Basisrelease, Annotation und Versuchsskripte lassen sich über bezeichnete Fassungen verbinden. Issues und Pull Requests ermöglichen Rückfragen und geprüfte Beiträge; die Versionierung macht die Untersuchung nachvollziehbar.

LOKALE NUTZUNG UND ÜBERGABE

Klappt es auch ohne Leas Rechner?

Ein Versuch ohne Zuruf

Jonas lädt den Release auf einen anderen Rechner und beginnt bei README. Das Beispiel erwartet noch Leas Netzlaufwerk. Im Team hatte das niemand bemerkt: Alle arbeiteten mit derselben Infrastruktur.

Der Einstieg wird verbessert

Lea ersetzt persönliche Pfade durch relative Pfade und dokumentiert Aufruf, Abhängigkeiten und erwartetes Ergebnis. Jonas startet noch einmal vom frischen Paket. Diesmal gelingt die erste Untersuchung ohne Hilfe. Vor dem Ende von Leas Stelle erprobt er auch die Anleitung zum nächsten Release.

MERKEN

Der Einstieg muss ohne die Projektumgebung funktionieren.

Wir liefern einen kleinen vollständigen Nutzungseinstieg und den gesamten bezeichneten Datenbestand. Ein anderer Rechner ist unser praktischer Gegencheck.

▼ Beispiel und Tipp

Beispiel: Statt eines Pfads zu Leas Netzlaufwerk sucht das Skript korpus.xml relativ zu seinem eigenen Ordner. Der Einstieg läuft mit Python-Standardbibliothek ohne API oder Konto.

Tipp: Dateinamen ohne betriebssystemspezifische Sonderzeichen und relative Pfade verwenden.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Dateinamen ohne betriebssystemspezifische Sonderzeichen und relative Pfade verwenden.
2. Einen Aufruf mit Abhängigkeiten und erwarteter Ausgabe in README dokumentieren.
3. Eine andere Person das frisch entpackte Paket ausführen lassen; Kontakt und Pflegezusage ergänzen.

Konkretes Beispiel

Statt eines Pfads zu Leas Netzlaufwerk sucht das Skript korpus.xml relativ zu seinem eigenen Ordner. Der Einstieg läuft mit Python-Standardbibliothek ohne API oder Konto.

Erledigt, wenn: Das dokumentierte Ergebnis entsteht in einer frischen Umgebung; fehlendes Wissen ist in der Anleitung ergänzt.

Git hilft dabei: Ein bezeichneter Release, vollständige lokale Daten, versionierte Beispiele und klare Änderungsnachweise ermöglichen selbstständige Nutzung. Die Anleitung für spätere Releases wird ebenfalls praktisch erprobt.

DOWNLOADS UND ZUSATZDIENSTE

Die Bibliothek wartet auf den Server.

Öffentlich und trotzdem nicht nutzbar

Elif möchte viele Rätsel lokal vergleichen. Die Bibliothek lädt sie einzeln über die API. Der Server drosselt die Anfragen; ihre Untersuchung bleibt stehen.

Ein unabhängiger Arbeitsweg

Das Team bietet den vollständigen Release zum Download an und lässt die Bibliothek auch lokale Dateien lesen. Eine kleine Probe führt zum ersten Ergebnis. API und Frontend bleiben zusätzliche Zugänge.

MERKEN

Korpusanalysen dürfen nicht vom laufenden Projektserver abhängen.

Unser Grundangebot sind versionierte lokale Daten und dokumentierte Exporte. Einen zusätzlichen Dienst richten wir erst für einen konkreten Bedarf und mit gesichertem Betrieb ein.

▼ Beispiel und Tipp

Beispiel: Zum Ausprobieren: nutzungsstart.zip. Für die vollständige Analyse: das bezeichnete Korpuspaket. Beide beschreiben ihre Dateien und den jeweiligen Umfang.

Tipp: Kleine Einstiegsdaten und vollständigen Release getrennt anbieten; große Bestände sinnvoll portionieren.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Kleine Einstiegsdaten und vollständigen Release getrennt anbieten; große Bestände sinnvoll portionieren.
2. Bibliothek und Beispiele auch mit lokalen Daten ausführbar machen.
3. API oder Frontend nur ergänzen, wenn Nutzen, Zuständigkeit und Betrieb geklärt sind.

Konkretes Beispiel

Zum Ausprobieren: nutzungsstart.zip. Für die vollständige Analyse: das bezeichnete Korpuspaket. Beide beschreiben ihre Dateien und den jeweiligen Umfang.

Erledigt, wenn: Die Untersuchung kann auch fortgesetzt werden, wenn die API ausfällt.

Git hilft dabei: Git versioniert Daten und Software. Es finanziert und betreibt keine zusätzlichen Dienste.

PORTABILITÄT UND ERHALT

Der Host zieht um. Die Forschung soll bleiben.

Mehr als Dateien umziehen

Beim Wechsel des Hosts sind die Git-Dateien vorhanden. Verknüpfte Diskussionen und Release-Anhänge fehlen jedoch. Einige Entscheidungen sind nur dort erklärt.

Den benötigten Kontext erhalten

Jonas übernimmt die notwendigen Plattforminformationen und hält fachliche Entscheidungen im Repo fest. Die Prüfungen laufen auch außerhalb der bisherigen Automatisierung. Archivierte Releases sichern die früheren Forschungsbezüge.

MERKEN

Git-Geschichte und Plattformbestand sind verschiedene Dinge.

Wir sichern Git, veröffentlichte Pakete und die für das Verständnis nötigen Entscheidungen. Der Neuaufbau wird anhand einer versionierten Anleitung geprüft.

▼ Beispiel und Tipp

Beispiel: Git überträgt Commits und Tags. Release-Anhänge, relevante Diskussionen und Diensteinstellungen werden zusätzlich übernommen oder nachvollziehbar dokumentiert.

Tipp: Git-Referenzen und benötigte Zusatzdateien sichern; relevante Issues und Entscheidungen exportieren oder dokumentieren.

SO WÜRDEN WIR ES MACHEN

Das können Sie übernehmen

1. Git-Referenzen und benötigte Zusatzdateien sichern; relevante Issues und Entscheidungen exportieren oder dokumentieren.
2. Prüfungen und Exporte ohne die alte CI-Umgebung ausführen.
3. Releasepakete, Zitiernachweise, Kontakt und Berechtigungen am neuen Ort prüfen.

Konkretes Beispiel

Git überträgt Commits und Tags. Release-Anhänge, relevante Diskussionen und Diensteinstellungen werden zusätzlich übernommen oder nachvollziehbar dokumentiert.

Erledigt, wenn: Die Dateien sind am neuen Ort verständlich und die dokumentierten Verfahren ausführbar.

Git hilft dabei: Git-Daten sind portabel; wissenschaftliche Archivierung und dokumentierte Verfahren begrenzen die Abhängigkeit von Plattformfunktionen.